机器学习
第 10 章　无监督学习

——当数据没有标签，机器该怎样自己发现规律？

2026 年 3 月 17 日

本章题目导览：你将挑战哪些核心问题？

在监督学习中，每条数据都有一个正确答案（标签）告诉模型”这是猫””这是狗”。但现实中，大量数据根本没有标签——海量的用户行为、基因序列、图片……人工标注代价极高。**无监督学习**的任务就是：在没有标签的情况下，从数据中自动发现结构和规律。本章 7 道题覆盖聚类与降维两大核心方向：

- **Q10-01（概念辨析）**：有监督 vs 无监督——两种学习范式的根本区别，聚类到底在做什么？
- **Q10-02（算法推导）**：K-means 工作流程手动推导——给定数据点，一步步演算质心更新过程，理解收敛本质。
- **Q10-03（对比分析）**：三种聚类算法横向对比——K-means、层次聚类、密度聚类各有什么适用场景和缺陷？
- **Q10-04（代码实践）**：K-means API 使用——`sklearn` 的 `KMeans` 参数解析，如何用代码完成一次完整的聚类分析？
- **Q10-05（评估指标）**：聚类模型评估——轮廓系数、WCSS、CH 指数分别衡量什么？肘部法如何选 K？
- **Q10-06（数学理解）**：奇异值分解（SVD）——矩阵分解的核心原理，为什么 SVD 能实现降维？
- **Q10-07（综合应用）**：主成分分析（PCA）——PCA 和 SVD 的关系，用 SVD 实现 PCA 降维的完整流程是什么？

1　第 10 章　无监督学习：聚类与降维

前面所有章节学的都是**监督学习**：每条训练数据都有一个”正确答案”作为老师，模型在老师的监督下学习如何从输入预测输出。

但现在换一种场景：

- 你有 100 万条用户的购物记录，没有人告诉你用户属于哪类群体；
- 你有一张 1024×1024 的高清图片，想压缩成更小的尺寸但不失真太多；
- 你有 500 维的基因表达数据，想在二维平面上看清楚哪些样本相似。

这类问题没有标签，无法用交叉熵损失来训练。我们需要**无监督学习**：让模型自己在数据中发现结构。

先建立一个总感觉：无监督学习在做什么

想象你是一个图书管理员，面前堆着一万本没有任何分类标签的书。

- **聚类**：把书摆来摆去，按内容相似度自动归堆——武侠小说堆一起，技术书堆一起，虽然没人告诉你怎么分，但你凭书的”相似感”把它们分开了。
- **降维**：书太多放不下，你决定用一句话概括每本书的精华——从厚厚的书变成一句核心摘要，用更少的信息保留最重要的内容。

无监督学习的核心：没有老师打分，模型自己发现数据内部的**结构**和**规律**。

Q10-01 有监督 vs 无监督——范式对比与聚类本质

题目描述：

理解无监督学习最重要的一步，是明白它和监督学习的根本区别，以及聚类这件事的本质目标。

问题：

(1) 填写下表，对比有监督学习与无监督学习的关键区别：

对比维度	有监督学习	无监督学习
训练数据	带标签的样本 $\{(x_i, y_i)\}$	_____
学习目标	学习从输入到标签的映射 $f : x \to y$	_____
典型任务	分类、回归	_____
评估方式	对比预测值与真实标签（如准确率）	_____

(2) 聚类（Clustering）的核心目标用一句话描述是什么？它应满足哪两个基本原则（关于簇内和簇间的相似度）？

(3) 以下哪些任务属于无监督学习？（多选）

A. 给邮件自动打”垃圾邮件/正常邮件”标签（已有历史标注数据）

B. 对电商平台的用户按行为特征自动分群，用于精准营销

C. 将高维医疗影像数据压缩到二维平面以便可视化

D. 根据历史房价数据预测未来房价

E. 在没有类别标注的新闻语料库中，自动发现潜在的话题分布

来源溯源：[文档第 10 章 10.1.1 聚类简介]

小白必读：为什么”发现结构”这么重要？

- **标签太贵了**：ImageNet 有 120 万张图片，每张都需要人工标注，代价极高。但互联网上有数十亿张无标签图片，无监督学习就是利用这些”免费数据”的工具。
- **很多时候不知道该分几类**：监督分类时，类别是预先定义好的。但聚类不需要预设类别，它可以帮你**发现数据中自然存在的分组**，这在探索性分析中非常有价值。
- **降维是数据分析的基础工具**：高维数据（几百维）直接分析很困难，降到 2-3 维后，人眼就能直接看出数据的结构——聚不聚集、有没有异常点。

参考答案与详细解析

(1) 对比表格填写：

对比维度	有监督学习	无监督学习
训练数据	带标签的样本 $\{(x_i, y_i)\}$	仅有特征，无标签 $\{x_i\}$
学习目标	学习从输入到标签的映射	发现数据内在结构（分布、分组、主要成分）
典型任务	分类、回归	聚类、降维、异常检测、密度估计
评估方式	对比预测值与真实标签	内部指标（轮廓系数等）或下游任务效果

(2) 聚类的核心目标：

将数据集划分为若干个簇（Cluster），使得**同一簇内的样本尽可能相似（簇内距离小），不同簇之间的样本尽可能不同（簇间距离大）**。

(3) 属于无监督学习的任务：B、C、E。

- A 是有监督分类（有历史标注）；
- D 是有监督回归（有历史房价作为标签）；
- B 是聚类（无预定义分群标签）；
- C 是降维可视化（无标签，发现数据结构）；
- E 是主题模型（无标注，发现话题分布）。

Q10-02 K-means 工作流程手动推导

题目描述：

K-means 是最经典的聚类算法之一。理解它的关键，是能够手动推演一次完整的迭代过程——从初始化质心，到分配簇，到更新质心，再到收敛判断。

题目数据：

平面上有 6 个数据点：$A(1,1)$, $B(1,3)$, $C(2,2)$, $D(8,1)$, $E(8,3)$, $F(9,2)$。设 $K=2$，初始质心为 $\mu_1=(1,1)$（即点 A），$\mu_2=(8,1)$（即点 D）。距离度量使用欧氏距离：$d(x,y)=\sqrt{(x_1-y_1)^2+(x_2-y_2)^2}$。

问题：

(1) **第一次迭代——分配步骤：**计算每个点到 μ_1,μ_2 的欧氏距离，将每个点分配到距离最近的质心所在的簇。填写下表：

数据点	到 $\mu_1=(1,1)$ 的距离	到 $\mu_2=(8,1)$ 的距离	分配到簇
$A(1,1)$	—	—	—
$B(1,3)$	—	—	—
$C(2,2)$	—	—	—
$D(8,1)$	—	—	—
$E(8,3)$	—	—	—
$F(9,2)$	—	—	—

(2) **第一次迭代——更新步骤：**根据第 (1) 步的分配结果，重新计算每个簇的质心（取簇内所有点坐标的均值）。

(3) **第二次迭代：**用更新后的质心重新分配各点，再次更新质心，判断算法是否已经收敛（质心不再变化）。

(4) K-means 的收敛条件是什么？它一定能找到全局最优解吗？为什么？

来源溯源：[文档第 10 章 10.1.2 K 均值聚类工作流程]

小白必读：K-means 的三个核心步骤

- **Step 1——初始化：**随机选 K 个点作为初始质心（或用 K-means++ 策略）。
- **Step 2——分配（E 步）：**把每个数据点分配给距它最近的质心，形成 K 个簇。
- **Step 3——更新（M 步）：**对每个簇，把簇内所有点的坐标取均值，得到新质心。
- **重复 Step 2 和 Step 3**，直到质心不再移动（或移动距离小于阈值）为止。

"K-means" 名字的含义：K 是簇的数量，"means" 就是" 均值"——新质心是簇内数据点的**均值**。

生活比喻：K-means 就像”自动选座位”

假设班里来了一批新同学，老师想把大家分成 K 组，但不知道怎么分：

- **初始化**：随机叫 K 个同学站到教室各个角落，作为”组长”。
- **分配步骤**：每个剩余同学走向离自己最近的组长，加入他的小组。
- **更新步骤**：每个小组重新找一个”重心位置”，让站在那里的人到组内所有成员距离之和最小——这个重心就是新的质心（往往不是原来的组长了）。
- **收敛**：等到没有人需要换组，也没有质心需要移动，分组就稳定下来了。

参考答案与详细解析

(1) 第一次迭代——分配步骤：

计算各点到两个质心的距离：

数据点	到 $\mu_1 = (1,1)$	到 $\mu_2 = (8,1)$	分配到簇
$A(1,1)$	0	7	簇 1
$B(1,3)$	2	$\sqrt{53} \approx 7.28$	簇 1
$C(2,2)$	$\sqrt{2} \approx 1.41$	$\sqrt{37} \approx 6.08$	簇 1
$D(8,1)$	7	0	簇 2
$E(8,3)$	$\sqrt{53} \approx 7.28$	2	簇 2
$F(9,2)$	$\sqrt{65} \approx 8.06$	$\sqrt{2} \approx 1.41$	簇 2

(2) 第一次迭代——更新步骤：

- 簇 1 包含 $A(1,1), B(1,3), C(2,2)$：

$$\mu_1^{\text{new}} = \left(\frac{1+1+2}{3},\ \frac{1+3+2}{3}\right) = \left(\frac{4}{3},\ 2\right) \approx (1.33,\ 2)$$

- 簇 2 包含 $D(8,1), E(8,3), F(9,2)$：

$$\mu_2^{\text{new}} = \left(\frac{8+8+9}{3},\ \frac{1+3+2}{3}\right) = \left(\frac{25}{3},\ 2\right) \approx (8.33,\ 2)$$

(3) 第二次迭代：

用新质心 $\mu_1 = (1.33, 2)$，$\mu_2 = (8.33, 2)$ 重新分配各点：

由于六个点的位置分布非常明显（左侧三点 vs 右侧三点），重新计算距离后，分配结果与第一次完全一致——簇 1 仍为 $\{A, B, C\}$，簇 2 仍为 $\{D, E, F\}$。质心不再移动，**算法在第二次迭代后收敛**。

(4) 收敛条件与全局最优：

K-means 的收敛与局限性

- **收敛条件**：质心在前后两次迭代中不再发生变化（或变化量小于设定阈值 `tol`）。理论上 K-means 一定会在有限步内收敛，因为每次迭代都不会增加目标函数（WCSS）的值。
- **不保证全局最优**：K-means 是贪心算法，只保证收敛到**局部最优解**。结果对初始质心的选取非常敏感——不同的初始质心可能导致完全不同的聚类结果。
- **解决方案**：使用 **K-means++** 初始化策略（倾向于选择彼此较远的初始质心），或多次随机初始化后取最优结果（`n_init` 参数）。

Q10-03 三种聚类算法横向对比

题目描述：

除 K-means 外，层次聚类和密度聚类（DBSCAN）也是常用的聚类方法。三种算法各有所长，理解它们的差异有助于在实际问题中选择合适的工具。

问题：

(1) 填写下表，完成三种聚类算法的对比：

算法	核心思想	主要局限性	需要预设 K？
K-means	迭代更新质心，最小化_____	对_____敏感，只能发现_____形状的簇	__
层次聚类	自底向上合并（凝聚型）或自顶向下分裂，形成_____	计算复杂度高：O(__)，无法处理超大规模数据	__
密度聚类 (DBSCAN)	寻找被低密度区域分隔的高密度区域，将密度相连的点归为一簇	对_____参数敏感，高维数据效果差	__

(2) DBSCAN 中有两个核心参数：ε（邻域半径）和 `MinPts`（核心点的最小邻居数）。请解释：

a. 什么是”核心点”、”边界点”和”噪声点”？

b. 如果把 ε 调得很大，会发生什么？如果 ε 很小呢？

(3) 下面四种数据分布，分别最适合用哪种聚类算法？请匹配并说明理由。

A. 三个球形簇，各簇大小相近，维度较低（如 2D）

B. 两个月牙形（非凸）簇，中间无明显间隔

C. 希望得到一棵完整的”树形层次结构”，可视化数据的多层级关系

D. 数据中含有大量异常点（噪声），簇的形状不规则

来源溯源：[文档第 10 章 10.1.2 层次聚类 / 密度聚类]

小白必读：为什么要有这么多种聚类算法？

- **K-means 的盲区**：K-means 假设每个簇是”球形的”，且大小相近。面对月牙形、环形等不规则簇，它会彻底失效——因为质心（均值点）根本无法代表非凸形状的中心。
- **层次聚类的优势**：不需要预设 K，生成的树状图（Dendrogram）保留了数据的完整层次关系，可以在任意层”切割”得到不同粒度的聚类结果。
- **DBSCAN 的独特能力**：天然支持发现任意形状的簇，并能自动识别噪声点（标记为 -1）。这对有噪声的真实数据非常宝贵——K-means 会把所有噪声点都强行分配到某个簇中。

参考答案与详细解析

(1) 对比表格填写：

算法	核心思想	主要局限性	需要预设 K？
K-means	迭代更新质心，最小化簇内平方和（WCSS）	对初始质心敏感，只能发现球形/凸形状的簇	是
层次聚类	自底向上合并，形成树状图（Dendrogram）	复杂度 $O(n^2)$ 到 $O(n^3)$，无法处理超大规模数据	否
DBSCAN	寻找高密度区域，密度相连点归一簇	对 ε、`MinPts` 参数敏感，高维数据效果差	否

(2) DBSCAN 核心概念：

- **核心点（Core Point）**：在半径 ε 内邻居数 $\geq$ `MinPts` 的点，是高密度区域的”骨干”。
- **边界点（Border Point）**：邻居数 $<$ `MinPts`，但在某个核心点的 ε 邻域内，是簇的”边缘成员”。
- **噪声点（Noise Point）**：既不是核心点，也不在任何核心点的邻域内，标记为异常值（标签 -1）。
- **ε 很大**：越来越多的点相互”密度可达”，不同的簇可能被合并成一个大簇，极端情况下所有点都在同一个簇里。
- **ε 很小**：大多数点找不到足够的邻居成为核心点，大量点被标记为噪声，簇数量极多且每个簇只含极少数点。

(3) 场景匹配：

- **A $\rightarrow$ K-means**：球形等大簇是 K-means 最适合的场景，计算效率高。
- **B $\rightarrow$ DBSCAN**：月牙形等非凸形状，K-means 无能为力，DBSCAN 能沿密度边界准确划分。
- **C $\rightarrow$ 层次聚类**：层次聚类生成完整的 Dendrogram，是可视化层次关系的首选。
- **D $\rightarrow$ DBSCAN**：DBSCAN 天然具备噪声识别能力，不规则形状也能处理。

Q10-04 K-means API 使用：sklearn.KMeans 参数解析

题目描述：

下面是一段使用 scikit-learn 的 `KMeans` 对鸢尾花数据集（只取前两个特征）进行聚类的完整代码，请结合代码回答问题。

```python
from sklearn.cluster import KMeans
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt
import numpy as np

# 加载数据，只取前两个特征
iris = load_iris()
X = iris.data[:, :2]   # shape: (150, 2)

# 标准化
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# 训练 KMeans 模型
kmeans = KMeans(
    n_clusters=3,
    init='k-means++',
    n_init=10,
    max_iter=300,
    random_state=42
)
kmeans.fit(X_scaled)

# 获取结果
labels = kmeans.labels_            # 每个样本的簇标签
centers = kmeans.cluster_centers_  # 簇质心坐标
inertia = kmeans.inertia_          # WCSS 值

```

问题：

(1) 解释 `KMeans` 构造函数中每个参数的含义：`n_clusters`、`init`、`n_init`、`max_iter`、`random_state`。

(2) 为什么在聚类之前要做**标准化**（`StandardScaler`）？如果不标准化，会发生什么问题？

(3) `kmeans.labels_` 的形状和取值范围是什么？`kmeans.cluster_centers_` 的形状是什么？

(4) 训练完毕后，如果有一个新数据点 `x_new = [[5.1, 3.5]]`，如何用训练好的模型预测它属于哪个簇？写出对应的代码。

来源溯源：[文档第 10 章 10.1.3 K-means API 使用]

小白必读：为什么聚类一定要标准化？

- **距离受量纲影响：** K-means 用欧氏距离衡量相似度。假设特征 1 的取值范围是 $[0, 1]$，特征 2 的取值范围是 $[0, 10000]$（如房价）。在计算距离时，特征 2 的差异会完全”碾压”特征 1——相当于特征 1 根本没有参与聚类。
- **标准化消除量纲：** `StandardScaler` 将每个特征变换为均值为 0、标准差为 1 的分布，使不同量纲的特征在距离计算中”站在平等地位”上。
- **经验法则：** 只要用到”距离”的算法（K-means、KNN、SVM、PCA……），都应该优先考虑是否需要标准化。

参考答案与详细解析

(1) 参数含义：

- `n_clusters=3`：聚成 $K = 3$ 个簇，即最终输出 3 个类别。
- `init='k-means++'`：使用 K-means++ 策略初始化质心，倾向于将初始质心选得彼此尽量分散，减少陷入差结果的概率（相比纯随机初始化更稳定）。
- `n_init=10`：用 10 种不同的随机初始质心各跑一次，取 WCSS 最小的结果作为最终输出，进一步降低局部最优风险。
- `max_iter=300`：单次运行的最大迭代次数，防止在边缘情况下无限循环。
- `random_state=42`：固定随机种子，保证结果可复现。

(2) 标准化的必要性：

欧氏距离对各特征的量纲极其敏感。鸢尾花数据中，花萼长度（约 4–8 cm）和花萼宽度（约 2–5 cm）量纲相似，但若混入花瓣长度（0.1–7 cm）且未标准化，数值范围差异会导致距离主要由范围大的特征决定，范围小的特征几乎失效。标准化后，所有特征均等地参与距离计算。

(3) 结果形状：

- `labels_`：形状 `(150,)`，取值范围 $\{0, 1, 2\}$（即 0 到 $K - 1$），每个元素是对应样本被分配的簇编号。
- `cluster_centers_`：形状 `(3, 2)`，即 K 个簇，每个簇的质心是一个 2 维向量（与输入特征维度一致）。

(4) 预测新数据点的代码：

```python
x_new = [[5.1, 3.5]]
# 必须先用同一个 scaler 做标准化！
x_new_scaled = scaler.transform(x_new)
# 预测所属簇
pred_label = kmeans.predict(x_new_scaled)
print(f"新数据点被分配到簇：{pred_label[0]}")

```

重要提醒：训练与预测时的标准化必须一致

训练时用 `fit_transform(X)`，预测新数据时只能用 `transform(x_new)`（不能再 `fit`）。原因：新数据必须用**训练集的均值和标准差**来缩放，才能和训练好的质心坐标处于同一空间中进行距离比较。

Q10-05 聚类模型评估：如何判断聚类效果好不好？

题目描述：

聚类没有真实标签，无法用准确率评估。但有一系列**内部评估指标**，从不同角度衡量聚类质量。本题覆盖四种主要评估方法。

问题：

(1) **轮廓系数（Silhouette Coefficient）：**

对样本 i，定义：

$$a(i) = \text{样本}i \text{ 到同簇内其他所有点的平均距离（簇内凝聚度）}$$

$$b(i) = \text{样本}i \text{ 到最近其他簇所有点的平均距离（簇间分离度）}$$

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

整体轮廓系数为所有样本 $s(i)$ 的均值，取值范围为 $[-1, 1]$。

请回答：

a. $s(i)$ 接近 $+1$、接近 0、接近 -1 时，各自意味着什么？

b. 轮廓系数**越大越好**还是越小越好？

(2) **簇内平方和（WCSS）与肘部法（Elbow Method）：**

WCSS 定义为：

$$\text{WCSS} = \sum_{k=1}^{K} \sum_{x_i \in C_k} \|x_i - \mu_k\|^2$$

a. 随着 K 增大，WCSS 的变化趋势是什么？为什么 WCSS 不能直接用于选最优 K（即为什么不能直接最小化 WCSS）？

b. 肘部法的思路是什么？”肘部”出现在哪里时，认为该 K 值是合理选择？

(3) **CH 指数（Calinski-Harabasz Index）：**

$$\text{CH}(K) = \frac{\text{BSS}/(K-1)}{\text{WSS}/(n-K)}$$

其中 BSS 是簇间方差（Between-Cluster Sum of Squares），WSS 是簇内方差。CH 指数**越大越好**还是越小越好？它衡量的核心思想是什么？

(4) 用 `sklearn` 计算这三个指标的代码分别是什么？（只需写出关键一行调用）

来源溯源：[文档第 10 章 10.1.4 聚类模型评估]

小白必读：为什么聚类评估比分类评估难？

- **没有”标准答案”对比**：分类可以直接算”预测是否等于真实标签”，而聚类只有数据本身，没有外部标签告诉我们分几类是”对的”。
- **评估指标衡量的是”结构好不好”**：轮廓系数看”同类聚不聚、异类散不散”；WCSS 看簇内的紧凑程度；CH 指数看两者的比值。不同指标从不同角度打分，往往需要综合参考。
- **K 的选取是一门”艺术”而非”科学”**：没有唯一正确答案，肘部法是经验性的，有时肘部不明显，需要结合业务知识判断。

参考答案与详细解析

(1) 轮廓系数解读：

- $s(i) \approx +1$：$b(i) \gg a(i)$，即该点离自己所在簇的其他点很近（内聚好），离最近的其他簇很远（分离好）——分配得很好。
- $s(i) \approx 0$：$b(i) \approx a(i)$，该点处于两个簇的边界附近，分配到哪个簇差别不大。
- $s(i) \approx -1$：$a(i) \gg b(i)$，该点到自己簇内的距离比到最近其他簇的距离还大——说明该点被分配到了错误的簇。

轮廓系数越大越好，理想值接近 $+1$。

(2) WCSS 与肘部法：

- **WCSS 随 K 增大单调递减**：$K = n$（每个点单独一个簇）时，WCSS $= 0$，但这无任何意义。不能直接最小化 WCSS，否则答案永远是”每个点自己一个簇”。
- **肘部法**：绘制 K 从 1 到若干取值时的 WCSS 曲线。随着 K 增大，WCSS 下降速度先快后慢，形成”肘部”形状。在下降速度明显变缓的”转折点”处，该 K 值被认为是数据中自然簇数的较好估计——继续增大 K 带来的收益已经很小，不值得增加模型复杂度。

(3) CH 指数：

CH 指数越大越好。其核心思想是衡量**簇间分散度与簇内紧凑度的比值**：分子（簇间方差大）意味着不同簇之间距离远，分母（簇内方差小）意味着同一簇内部紧凑——这正是好聚类的两个条件。

(4) sklearn 关键代码：

```python
from sklearn.metrics import silhouette_score, calinski_harabasz_score

# 轮廓系数（越大越好，范围 [-1, 1]）
sil = silhouette_score(X_scaled, labels)

# CH 指数（越大越好）
ch = calinski_harabasz_score(X_scaled, labels)

# WCSS（通过 KMeans 的 inertia_ 属性直接获取）
wcss = kmeans.inertia_

```

Q10-06 奇异值分解（SVD）：矩阵分解的核心原理

题目描述：

奇异值分解（Singular Value Decomposition, SVD）是线性代数中最重要的矩阵分解方法之一，也是 PCA 降维的数学基础。理解 SVD 的结构是掌握降维的关键。

题目背景：

对任意实矩阵 $A \in \mathbb{R}^{m\times n}$，SVD 将其分解为：

$$A = U\Sigma V^T$$

其中：

- $U \in \mathbb{R}^{m\times m}$：左奇异向量矩阵（列向量两两正交，$U^TU = I$）
- $\Sigma \in \mathbb{R}^{m\times n}$：对角矩阵，对角元素 $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r \geq 0$ 称为奇异值
- $V \in \mathbb{R}^{n\times n}$：右奇异向量矩阵（列向量两两正交，$V^TV = I$）

问题：

(1) 矩阵 A 的”秩”（Rank）与奇异值之间有什么关系？奇异值为 0 意味着什么？

(2) **截断 SVD 实现降维：**若只保留最大的 k 个奇异值（$k < r$），得到近似矩阵：

$$A_k = U_k\Sigma_kV_k^T$$

其中 $U_k \in \mathbb{R}^{m\times k}$，$\Sigma_k \in \mathbb{R}^{k\times k}$，$V_k \in \mathbb{R}^{n\times k}$。

a. A_k 的形状是什么？它是 A 的精确复原还是近似复原？

b. 为什么保留”最大的 k 个奇异值”能保留最多信息？（奇异值大小代表什么？）

(3) 设 $A = \begin{pmatrix} 3 & 0 \\ 0 & 2 \\ 0 & 0 \end{pmatrix}$，直接写出其 SVD 分解结果（$U, \Sigma, V^T$），无需推导。（提示：$A$ 已经是接近对角形式）

(4) 用 NumPy 进行 SVD 分解的代码是什么？如何只保留前 $k = 2$ 个奇异值来实现近似重建？

来源溯源：[文档第 10 章 10.2.1 奇异值分解]

生活比喻：SVD 就像”拍照压缩”

你拍了一张 1000×1000 的图片（100 万个像素值的矩阵 A）：

- **完整 SVD：**把图片”拆解”成一系列”基础图案”的叠加，每个基础图案由一对奇异向量定义，叠加的”权重”就是奇异值。
- **奇异值大 = 基础图案重要：**奇异值大的那几个”基础图案”贡献了图片的主要结构（轮廓、主色调）；奇异值小的贡献的是细节和噪声。
- **截断 SVD = 有损压缩：**只保留最重要的 k 个基础图案，用 $A_k \approx A$ 来存储图片。k 越小，文件越小，但图片越模糊；k 越大，越接近原图，但存储空间也越大。

参考答案与详细解析

(1) 秩与奇异值的关系：

矩阵 A 的秩 r 等于非零奇异值的个数：$r = \text{rank}(A) = |\{i : \sigma_i > 0\}|$。奇异值为 0 意味着该方向上矩阵没有" 信息量"（数据在该方向上的投影为零），对应的奇异向量对矩阵的重建没有贡献。

(2) 截断 SVD：

a. $A_k = U_k \Sigma_k V_k^T$ 的形状仍为 $\mathbb{R}^{m\times n}$（与原矩阵相同）。它是**近似复原**而非精确复原——用更少的参数（k 个奇异值及对应向量）近似表示原矩阵，$k < r$ 时会有信息损失。

b. 奇异值 σ_i 的大小代表矩阵在该奇异向量方向上的" 能量"（方差贡献）。最大的奇异值对应最能解释数据变化的方向，保留大奇异值意味着保留了原矩阵中最主要的结构信息，舍弃小奇异值相当于去除噪声或次要细节。

(3) 简单矩阵的 SVD：

$A = \begin{pmatrix} 3 & 0 \\ 0 & 2 \\ 0 & 0 \end{pmatrix}$ 已为列正交形式，其 SVD 可直接写出：

$$U = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}, \quad \Sigma = \begin{pmatrix} 3 & 0 \\ 0 & 2 \\ 0 & 0 \end{pmatrix}, \quad V^T = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$$

奇异值为 $\sigma_1 = 3$，$\sigma_2 = 2$，秩 $r = 2$。

(4) NumPy SVD 代码：

```python
import numpy as np

A = np.array([[3, 0], [0, 2], [0, 0]], dtype=float)

# 完整 SVD 分解
U, s, Vt = np.linalg.svd(A, full_matrices=True)
# U: (3,3)  s: (2,) 奇异值向量  Vt: (2,2)

# 只保留前 k=2 个奇异值进行近似重建
k = 2
U_k = U[:, :k]          # (3, 2)
S_k = np.diag(s[:k])    # (2, 2)
Vt_k = Vt[:k, :]        # (2, 2)

A_k = U_k @ S_k @ Vt_k   # 近似重建，形状仍为 (3, 2)
print("重建误差:", np.linalg.norm(A - A_k))

```

Q10-07 主成分分析（PCA）：用 SVD 实现降维的完整流程

题目描述：

主成分分析（Principal Component Analysis, PCA）是最常用的降维方法，其核心可以用 SVD 来实现。本题通过一个完整的数学推导和代码实践，理解 PCA 的本质。

题目背景：

设数据矩阵 $X \in \mathbb{R}^{n\times p}$（$n$ 个样本，p 个特征）。PCA 的目标是找到一组新的低维坐标轴（主成分），使数据在这些坐标轴上的**方差最大化**（信息保留最多）。

SVD 实现 PCA 的步骤：

1. **中心化**：$X_c = X - \bar{X}$（每列减去对应特征的均值）
2. **对 X_c 做 SVD**：$X_c = U\Sigma V^T$
3. **主成分方向**：V 的列向量（即 V^T 的行向量）就是主成分方向
4. **降维投影**：$Z = X_c V_k$（保留前 k 个主成分，$Z \in \mathbb{R}^{n\times k}$）

问题：

(1) 为什么 PCA 第一步必须**中心化**（均值归零）？不中心化会发生什么？

(2) ”方差最大化”和”信息保留最多”为什么是同一回事？请用直觉解释（不需要数学推导）。

(3) 在 PCA 中，如何判断降到 k 维后**保留了多少原始信息**？（用奇异值表示）给出公式。

(4) 下面是用 SVD 实现 PCA 并降维鸢尾花数据的代码，请填写空白处：

```python
import numpy as np
from sklearn.datasets import load_iris

iris = load_iris()
X = iris.data   # shape: (150, 4)

# Step 1: 中心化
X_c = X - X.mean(axis=____)   # 填写 axis 参数

# Step 2: SVD 分解
U, s, Vt = np.linalg.svd(X_c, full_matrices=False)

# Step 3: 降到 2 维（保留前 2 个主成分）
k = 2
Z = X_c @ Vt[:k, :].____   # 填写转置操作

# Step 4: 计算前 2 个主成分解释的方差比例
explained_ratio = (s[:k]**2).sum() / ____   # 填写分母

print(f"降维后形状: {Z.shape}")
print(f"前2个主成分解释方差比例: {explained_ratio:.2%}")

```

(5) PCA 中的**主成分**（Principal Component）在几何上代表什么？第一主成分方向和数据方差最大方向有什么关系？

来源溯源：[文档第 10 章 10.2.2 主成分分析]

小白必读：PCA 到底在做什么？

- **本质是”换一套坐标系”**：原来的坐标轴是人为定义的（身高、体重、年龄……），主成分轴是数据自己告诉我们的——沿着数据变化最大的方向。
- **投影 = 在新坐标系下读坐标**：$Z = X_c V_k$ 的含义是：把每个数据点投影到前 k 个主成分方向上，得到新的低维坐标。
- **为什么要降维？**：4 维的鸢尾花数据很难直接可视化，降到 2 维后，散点图上就能清晰看到三个品种的分布，发现聚类结构。高维特征中往往存在大量冗余（相关的特征），PCA 能把这些冗余压缩掉，只保留最有信息量的方向。

生活比喻：PCA 就像”找最佳拍照角度”

你有一个 3D 雕塑，想拍一张 2D 照片，让看照片的人尽可能理解雕塑的形状：

- **随意拍（随机降维）**：可能正对着雕塑最薄的一面，看起来像一条线，信息丢失严重。
- **PCA 拍法**：自动找到能让雕塑”展开得最大”的角度——从这个角度看，雕塑的细节、轮廓、层次感保留最多。
- **第一主成分**：信息最丰富的那个维度（雕塑展开最大的方向）；第二主成分：在与第一主成分垂直的方向中，信息最丰富的那个；以此类推。

PCA 的精髓：用更少的维度，保留最多的”信息量”（方差）。

参考答案与详细解析

(1) 中心化的必要性：

若不中心化，SVD 的结果会”偏向”数据均值的方向——第一个奇异向量会主要指向数据的均值方向，而非真正的最大方差方向。中心化的目的是消除均值的影响，让 SVD 纯粹分析数据的**离散/变化结构**，而非数据的位置。

(2) 方差最大化 = 信息保留最多：

如果数据在某个方向上的投影方差很大，说明不同样本在这个方向上有显著差异——这个差异就是可以用来区分样本的信息。如果投影方差接近零，所有样本投影到这个方向上几乎得到同一个值，这个方向传达不了任何样本间的区别，相当于没有信息。因此，保留方差最大的方向 = 保留最多能区分样本的信息。

(3) 方差解释比例公式：

前 k 个主成分解释的方差比例为：

$$\text{Explained Variance Ratio} = \frac{\sum_{i=1}^{k} \sigma_i^2}{\sum_{i=1}^{r} \sigma_i^2}$$

其中 σ_i 是第 i 个奇异值，r 是非零奇异值的个数（即矩阵的秩）。奇异值的平方 σ_i^2 正比于数据在第 i 个主成分方向上的方差。

(4) 代码填空答案：

```python
# Step 1: 中心化
X_c = X - X.mean(axis=0)    # axis=0 表示对每列（每个特征）求均值

```

```python
# Step 3: 降到 2 维
Z = X_c @ Vt[:k, :].T       # Vt[:k,:].T 形状为 (4, 2), 投影到 2 维

# Step 4: 方差解释比例
explained_ratio = (s[:k]**2).sum() / (s**2).sum()    # 分母是全部奇异值的平方和

```

运行结果说明：鸢尾花 4 维数据降到 2 维后，通常前 2 个主成分可以解释约 97% 的方差，说明降维几乎没有丢失有用信息。

(5) 主成分的几何含义：

主成分的几何解释

- **主成分方向：** V 的列向量，彼此正交（垂直）的单位向量，构成新的坐标轴。
- **第一主成分（PC1）：** 数据点在该方向上的投影方差最大，即这个方向是数据" 变化最剧烈" 的方向。
- **第二主成分（PC2）：** 在与 PC1 垂直的所有方向中，投影方差最大的方向。
- **降维的本质：** 把原来 p 维坐标系旋转为以主成分为轴的新坐标系，然后只保留前 k 个轴的坐标，丢弃方差最小（信息最少）的若干维度。

现实应用场景

PCA 和聚类的经典组合——人脸识别、基因表达分析：

在人脸识别中，原始图片可能是 $128 \times 128 = 16384$ 维的向量，直接聚类或分类效率极低。先用 PCA 降到 100 维，再做 K-means 聚类或分类，可以在大幅降低计算量的同时保留 99% 以上的信息。这种" 先 PCA 降维、再聚类/分类" 的两阶段流程，是工业界数据分析的经典 pipeline。SVD 在推荐系统中也扮演核心角色——Netflix、Amazon 的协同过滤算法，本质上就是对用户-商品评分矩阵进行截断 SVD，从中挖掘用户偏好的隐含特征。

2 本章小结：无监督学习的两条主线

学完本章 7 道题，可以把无监督学习的核心浓缩成两条主线：

- **聚类主线：** 数据有内在分组结构 → K-means（球形簇，需要预设 K）/ 层次聚类（树状结构，可视化好）/ DBSCAN（任意形状，能识别噪声）→ 用轮廓系数、WCSS、CH 指数评估聚类质量 → 肘部法辅助选 K。
- **降维主线：** 高维数据冗余多 → SVD 将矩阵分解为奇异值和奇异向量 → 截断 SVD 只保留大奇异值方向 → PCA 通过对中心化数据做 SVD 找到方差最大方向 → 降维后保留最多信息。

给零基础读者的一句总比喻

如果把数据比作一群人在广场上的站位：

- **聚类**：不给任何提示，让你把自然地聚在一起的人归为一组——K-means 用圆圈圈，DBSCAN 用"密度围栏"，层次聚类画一棵家谱树。
- **PCA 降维**：广场是 3D 的，但你只有一张 2D 纸，选一个最能体现人群分布的角度拍下来——PCA 帮你找到这个最佳角度，让拍出来的平面图损失最少信息。
- **两者结合**：先 PCA 降维（消除噪声、减少冗余维度），再聚类（在低维空间中结构更清晰），是无监督分析的黄金搭档。